

第8章

AXI 总线接口设计

从这一章开始，我们将进入一个新的阶段——为设计出的 CPU 增加 AXI 总线接口。在大多数真实的计算机系统中，CPU 通过总线与系统中的内存、外设进行交互。没有总线，CPU 就是个“光杆司令”，什么工作也做不了。总线接口可以自行定义，也可以遵照工业界的标准。显然，遵照工业界的标准有助于与大量第三方的 IP 进行集成。因此，本书选用 AMBA AXI 总线协议作为 CPU 总线接口的协议规范。

本章的设计任务有两个难点：一是 CPU 内部要如何调整以适应总线接口下的访存行为，二是如何设计出一个遵循 AXI 总线协议的接口。为了降低设计的难度，我们按照工程实践的经验，将这部分设计工作划分为三个阶段：

- 阶段一：将原有 CPU 访问 SRAM 的接口调整为类 SRAM 总线接口。类 SRAM 总线只是在 SRAM 接口的基础上增加了握手信号，可以降低直接实现 AXI 总线的设计复杂度。
- 阶段二：设计实现一个“类 SRAM-AXI”的转接桥，拼接上阶段一完成的 CPU，运行 AXI 固定延迟验证。读者将从这个阶段开始学习并实现 AXI 总线协议。
- 阶段三：完善阶段二的 CPU，完成 AXI 随机延迟验证。

【本章学习目标】

- 理解片上总线的一般性原理。
- 掌握总线接口与 CPU 内部流水线之间的交互设计。

【本章实践目标】

本章有三个实践任务，请读者在学习完本章内容后，依次完成。

- 本章 8.1 节和 8.2 节的内容对应实践任务一。
- 本章 8.3 节和 8.4 节的内容对应实践任务二和实践任务三。

8.1 类 SRAM 总线

我们为什么要定义一套类 SRAM 的总线协议呢？出发点有两个：

- 1) 一部分初学者完全不知道如何从现有取指和访存的 SRAM 接口改出 AXI 接口。
 - 2) 一部分初学者过于激进地使用 AXI 协议的特性, 把设计改得太复杂, 出现大量错误。
- 如果用一句话来介绍这套类 SRAM 总线协议, 那就是: 添加了握手机制的 SRAM 接口。

8.1.1 主方和从方

首先, 总线是处理器和内存、外设交互的通道, 交互行为具体体现为读和写两种类型的操作。既然是交互, 至少要有两个参与主体, 我们将发起方称为“主方”(Master), 响应方称为“从方”(Slave)。对于读操作来说, 主方提出读请求, 从方接收请求并返回数据; 对于写操作来说, 主方提出写请求并发出数据, 从方接收请求和数据。

8.1.2 类 SRAM 总线接口信号的定义

表 8-1 中列出了类 SRAM 总线接口信号的说明。

表 8-1 类 SRAM 总线接口信号

信号	位宽	方向	功能
clk	1	input	时钟
req	1	master → slave	请求信号, 为 1 时有读写请求, 为 0 时无读写请求
wr	1	master → slave	为 1 表示该次是写请求, 为 0 表示该次是读请求
size	[1:0]	master → slave	该次请求传输的字节数, 0: 1byte; 1: 2bytes; 2: 4bytes
addr	[31:0]	master → slave	该次请求的地址
wstrb	[3:0]	master → slave	该次写请求的字节写使能
wdata	[31:0]	master → slave	该次写请求的写数据
addr_ok	1	slave → master	该次请求的地址传输 OK, 读: 地址被接收; 写: 地址和数据被接收
data_ok	1	slave → master	该次请求的数据传输 OK, 读: 数据返回; 写: 数据写入完成
rdata	[31:0]	slave → master	该次请求返回的读数据

上表中除了用黑体标记的信号外, 其余信号都与原有的 SRAM 接口信号一一对应。其中, req 对应 en, wr 对应 ($\overline{\text{wen}}$), wstrb 对应 wen。存在对应关系的信号的含义没有任何改变, 因此不再解释。我们重点说明新增的 size、addr_ok、data_ok 三个信号。

对于 size 信号, 因为 AXI 总线协议上有 arsize 和 awsize 信号, 所以需要把这个信号通过类 SRAM 接口传送给 AXI 接口。类 SRAM 接口中 size 信号和不同访存操作之间的对应关系如表 8-2 所示 (需要注意表中标注黑体的部分: LWL/SWL 指令送到类 SRAM 接口上的地址的低两位需要抹为 0)。

表 8-2 类 SRAM 总线中 size 与访存操作的对应关系

流水线里的指令	计算得到的地址	送到类 SRAM 的地址	size	含义
取指	addr[1:0]==0	addr[1:0]==0	2	访问 4 字节
LW、SW	addr[1:0]==0	addr[1:0]==0	2	访问 4 字节
LH、LHU、SH	addr[1:0]==0	addr[1:0]==0	1	访问 2 字节

(续)

流水线里的指令	计算得到的地址	送到类 SRAM 的地址	size	含义
LH、LHU、SH	addr[1:0]==1	addr[1:0]==1	1	访问 2 字节
LB、LBU、SB	addr[1:0]==0	addr[1:0]==0	0	访问 1 字节
LB、LBU、SB	addr[1:0]==1	addr[1:0]==1	0	访问 1 字节
LB、LBU、SB	addr[1:0]==2	addr[1:0]==2	0	访问 1 字节
LB、LBU、SB	addr[1:0]==3	addr[1:0]==3	0	访问 1 字节
LWL、SWL	addr[1:0]==0	addr[1:0]==0	0	访问 1 字节
LWL、SWL	addr[1:0]==1	addr[1:0]==0	1	访问 2 字节
LWL、SWL	addr[1:0]==2	addr[1:0]==0	2	访问 4 字节, 实际使用 3 字节
LWL、SWL	addr[1:0]==3	addr[1:0]==0	2	访问 4 字节
LWR、SWR	addr[1:0]==0	addr[1:0]==0	2	访问 4 字节
LWR、SWR	addr[1:0]==1	addr[1:0]==1	2	访问 4 字节, 实际使用 3 字节
LWR、SWR	addr[1:0]==2	addr[1:0]==2	1	访问 2 字节
LWR、SWR	addr[1:0]==3	addr[1:0]==3	0	访问 1 字节

另外, 对于写事务, size 和 addr[1:0] 与 wstrb 是有对应关系的, 小尾端下的对应关系如表 8-3 所示 (对于 “size=2, addr=0” 的情况, wstrb 有两种可能取值: 0xf 和 0x7)。

表 8-3 类 SRAM 总线中 size、addr 与 wstrb 的对应关系

size 和 addr 取值	data[31:24]	data[23:16]	data[15:8]	data[7:0]	wstrb
size=0, addr=0	—	—	—	valid	0b0001
size=0, addr=1	—	—	valid	—	0b0010
size=0, addr=2	—	valid	—	—	0b0100
size=0, addr=3	valid	—	—	—	0b1000
size=1, addr=0	—	—	valid	valid	0b0011
size=1, addr=2	valid	valid	—	—	0b1100
size=2, addr=0	valid	valid	valid	valid	0b1111
	—	valid	valid	valid	0b0111
size=2, addr=1	valid	valid	valid	—	0b1110

addr_ok 信号用于和 req 信号一起完成读写请求的握手。只有在 clk 的上升沿同时看到 req 和 addr_ok 为 1 的时候才是一次成功的请求握手。

data_ok 信号有双重身份。对应读事务的时候, 它是数据返回的有效信号; 对应写事务的时候, 它是写入完成的有效信号。无论 data_ok 表达的是对读事务的响应还是对写事务的响应, 统称为数据响应。在类 SRAM 接口中, Master 对于数据响应总是可以接收, 所以不再设置 Master 接收 data_ok 的握手信号。也就是说, 如果存在未返回数据响应的请求, 则在 clk 的上升沿看到 data_ok 为 1 就可以认为是一次成功的数据响应握手。

8.1.3 类 SRAM 总线的读写时序

图 8-1 和图 8-2 分别展示了类 SRAM 总线上一次读事务和一次写事务的时序关系。

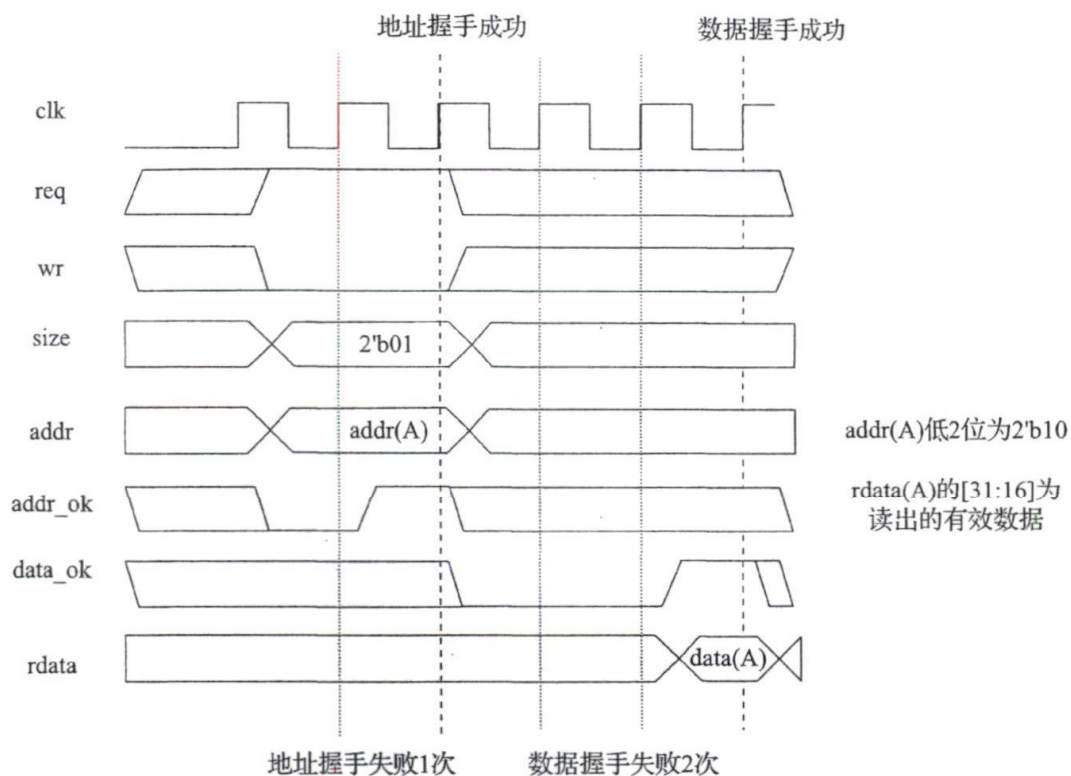


图 8-1 类 SRAM 总线上的一次读事务

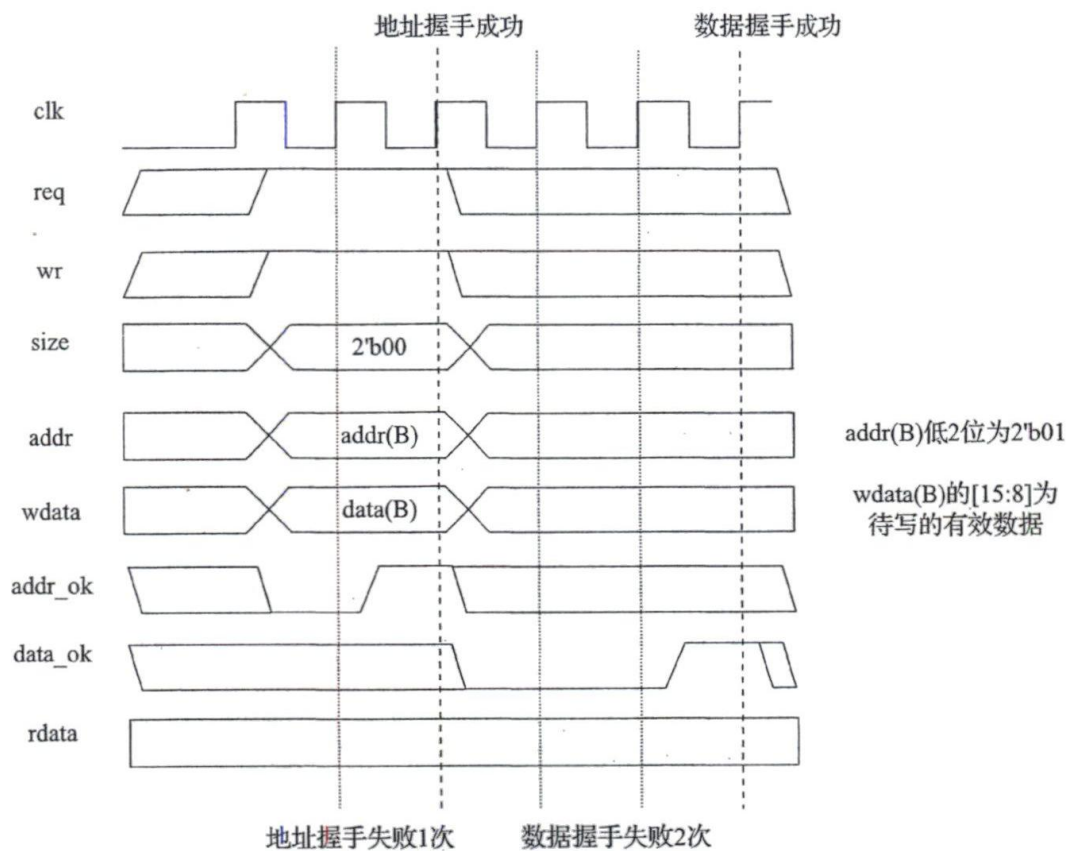


图 8-2 类 SRAM 总线上的一次写事务

图 8-3 给出了类 SRAM 总线上的连续写读的时序关系。连续写读时，从方返回的 `data_ok` 是严格按照请求发出的顺序返回的。在这幅图中，因为先发出写请求后发出读请求，所以必定先返回写事务的 `data_ok`，再返回读事务的 `data_ok`。但是，在一次读或写事务的请求握手成功之后至其响应返回之前，有可能再多次完成其他读、写事务的请求握手。也就是说，在接口的信号上，有可能出现 $(req1 \& addr_ok) \rightarrow (req2 \& addr_ok) \rightarrow (req3 \& addr_ok) \rightarrow (req4 \& addr_ok) \rightarrow \dots \rightarrow data_ok1$ 这样的握手信号序列。在考虑设计的时候，这种已发出请求但尚未响应的事务越多，设计就越复杂。要想控制这类事务的数目以简化设计，可以在 Master 端通过拉低 `req` 信号来暂停发送新事务的请求，在 Slave 端则可以通过拉低 `addr_ok` 信号来暂停接收新事务的请求。

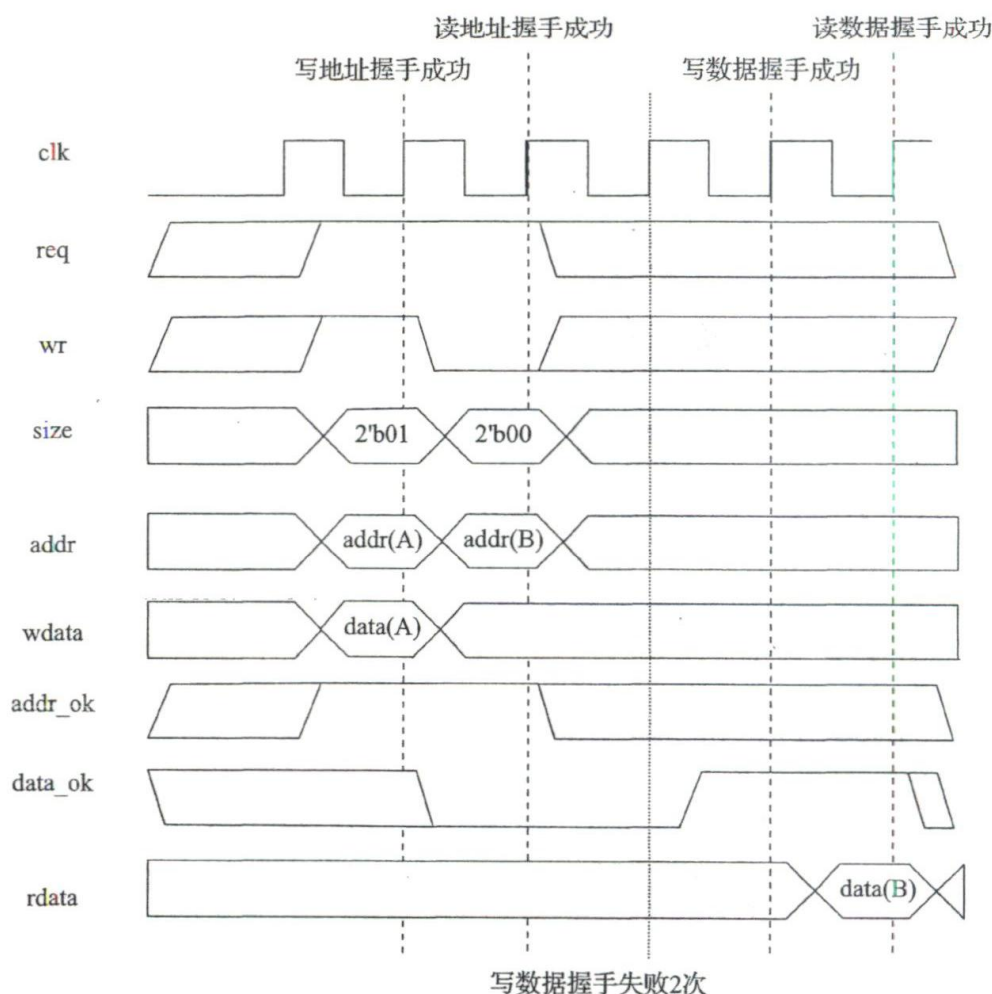


图 8-3 类 SRAM 总线上的连续写读事务

图 8-4 给出了类 SRAM 总线连续读写的时序关系示意。请注意，当 `addr_ok` 和 `data_ok` 同时有效时，它们各自对应不同的总线事务：`addr_ok` 表示当前传输事务的请求握手成功，`data_ok` 表示之前传输事务的数据响应握手成功。另外，图中读数据响应握手成功是有可能在写请求握手成功前完成的。

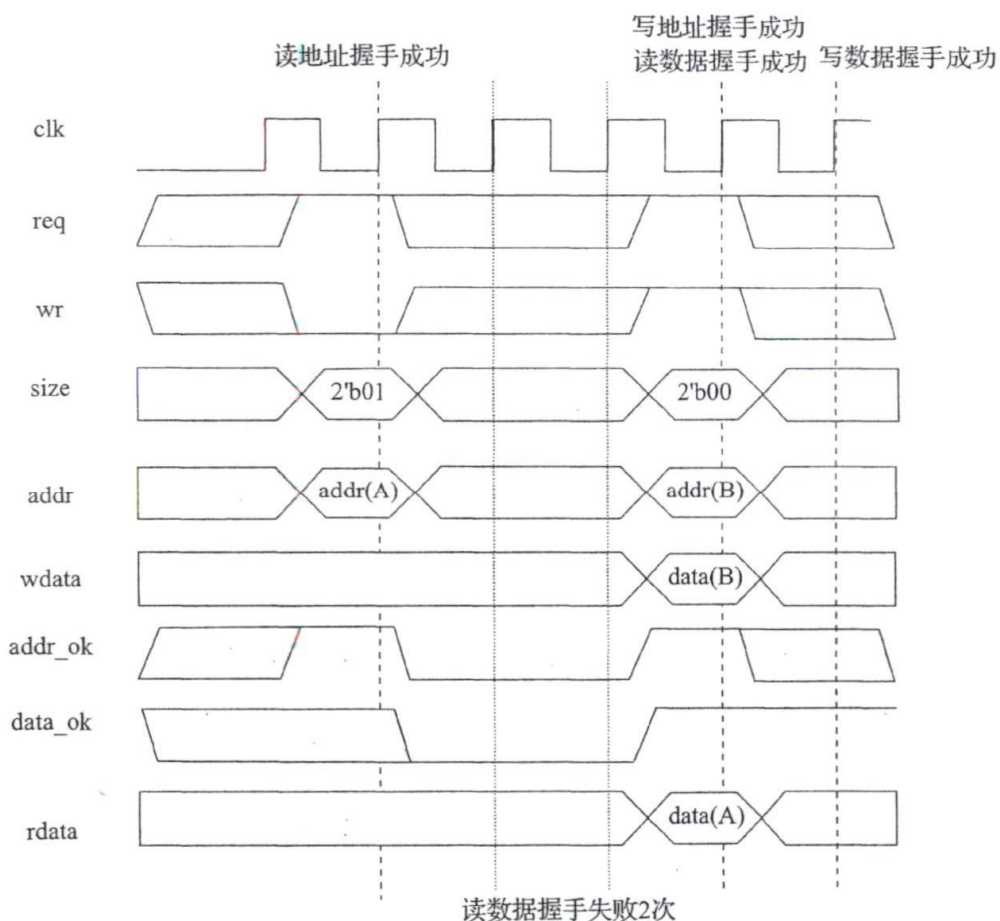


图 8-4 类 SRAM 总线连续读写事务

8.1.4 类 SRAM 总线的约束

为降低类 SRAM 总线的设计复杂度，我们对类 SRAM 总线做出以下约束：

1) 从方发出的 $addr_ok$ 的值不能依赖于主方发起的 req 的值。也就是说，生成 $addr_ok$ 的逻辑中不能看 req 信号。

2) 在 req 为 1 且 $addr_ok$ 为 0 时，允许主方更改 wr 、 $size$ 、 $addr$ 、 $wstrb$ 和 $wdata$ 。也就是说，类 SRAM 总线运行地址请求置起但未被接收时，可以更换请求类型和地址。这一点和后续要介绍的 AXI 总线不一样，AXI 总线要求：主方一旦发起某一地址或数据的传输，在该传输握手成功前，不得更改传输的地址或数据。

8.2 类 SRAM 总线的设计

在现有的 CPU 中，取指和访存部分使用的是标准的 SRAM 接口，我们需要将其改造成类 SRAM 总线接口。从 8.1 节的分析中我们知道，将标准 SRAM 接口改造为类 SRAM 接口只需要增加 3 个信号： $size$ 、 $addr_ok$ 和 $data_ok$ 。

在要增加的 3 个信号中，size 信号的生成非常简单，根据请求的属性直接生成即可（注意，对于 LWL/SWL 指令，需要对地址低 2 位做抹零的处理）。我们需要重点讨论的是 addr_ok 和 data_ok 这两个信号。

在现有的 CPU 中，取指或者访存阶段对于 SRAM 的访问是放在相邻两个流水级完成的，而且都是前一个流水级发出请求，后一个流水级接收响应。这种流水线划分的格局是不需要调整的，需要调整的是各流水级状态的控制逻辑。

8.2.1 取指设计的考虑

我们先来看取指阶段应该做出哪些调整。

1. 考虑 ready_go

基于我们提供的参考设计，取指地址请求是在 pre-IF（生成 nextPC）这个伪流水级发出的，指令返回是在 IF 流水级完成的。

首先，pre-IF 也需要维护一个 ready_go 信号。当从指令 RAM 取指时，pre-IF 伪流水级发出的请求总是能被接收，所以 pre-IF 指令的 ready_go 只需要关注 br_bus 上的 br_stall（不看 br_stall 也正确，参见 4.5.2 节的内容）。但是现在情况不同了，从类 SRAM 总线反馈回来的 addr_ok 未必时刻为 1。如果 addr_ok 为 0，意味着取指地址请求并没有被 CPU 外部接收。由于指令在 pre-IF 这级流水要做的处理就是发请求，既然请求都没有被接收，那么 ready_go 自然就是 0。仅当 req & addr_ok 置为 1 的时候，ready_go 才能置为 1。

对于 IF 流水级来说，原本从指令 RAM 取指时，请求接收的下一拍开始指令码就一定能够返回了，所以指令在 IF 流水级的唯一任务——拿到指令码——也能顺利完成，因此 ready_go 恒为 1。当我们引入类 SRAM 总线之后，情况就复杂了，只有 data_ok 返回 1 的时候，指令码才真正出现在接口上，也只有在这种情况下 IF 这一级的 ready_go 信号才能置为 1。

总结而言，取指地址请求和指令码返回这两个动作都需要进行握手，其对取指阶段两个流水级的影响体现在 pre-IF 级和 IF 级的 ready_go 信号上。是不是再次感受到了流水线控制信号 ready_go “包治百病”的神奇功效？

2. 考虑 allowin

上一小节中只是考虑了 pre-IF 和 IF 级的 ready_go 信号，对于流水线逐级互锁控制机制，还需要考虑 IF 级和 ID 级的 allowin 信号。

我们先考虑 pre-IF 级。pre-IF 级生成 nextPC，并对外发起取指的地址请求，等 addr_ok 来置 ready_go，当 ready_go 为 1 且 IF 级 allowin 为 1 时，pre-IF 级的指令流向 IF 级，pre-IF 级维护下一条指令的取指地址请求。根据 pre-IF-ready_go 和 IF-allowin 的组合情况，有 4 种可能：

1) pre-IF-ready_go=0, IF-allowin=0：显然，pre-IF 级继续发地址请求即可。

2) pre-IF-ready_go=0, IF-allowin=1：显然，和第 1 种情况一样，pre-IF 级继续发地址请求即可。

3) pre-IF-ready_go=1, IF-allowin=1: 表明 pre-IF 级接收到 addr_ok 且正好 IF 级 allowin 为 1, 当前指令就流进 IF 级, 此情况也很简单。

4) pre-IF-ready_go=1, IF-allowin=0: 表明 pre-IF 级接收到 addr_ok 但是 IF 级 allowin 为 0, 这种情况最复杂。

上述前 3 种情况比较简单, 第 4 种情况最复杂, 它会导致两个问题, 我们详细讨论一下。

● 问题一

当“pre-IF-ready_go=1, IF-allowin=0”时, pre-IF 级能不能把 req 信号置起来(也就是继续发该 PC 的取指地址请求)? 显然不能, 因为在这一拍, 类 SRAM 接口上的 req 和 addr_ok 同时为 1, 对于 CPU 外部来说, 这个请求已经被接收, 如果下一拍再置起 req, 类 SRAM 总线会把它当作一个新请求去处理。CPU 外部接收了多少个读请求, 就会一个不少地返回同样数目的数据。除非你非常清楚地知道这一点, 然后严格地过滤掉不需要的数据, 否则你一定会在 IF 级把指令码和 PC 的对应关系弄乱。典型的错误现象是, 你会看到连续执行的几条指令 PC 轨迹正确, 但是指令却一样。所以, 一定要注意, 如果地址请求被接口响应了, 但是指令无法在下一拍进入下一级, 那么从下一拍开始就不要再发同一个地址请求了。

● 问题二

当“pre-IF-ready_go=1, IF-allowin=0”时, pre-IF 级的地址请求已经被外部接收, 正在等 IF-allowin 的值, 外部随时可能返回 pre-IF 级取回的指令。如果在 IF-allowin 为 1 之前就收到了外部返回的 pre-IF 级取指的数据, 怎么办? 显然, 此时 pre-IF 级取回的指令无法进入 IF 级, 并且该指令只会在类 SRAM 总线接口的 rdata 端维持一拍。如果我们选择丢弃当前拍返回的指令, 就要让 pre-IF 级重新发起地址请求, 外部对一次请求只会返回一次数据, 如果不重新发请求, 外部就不会重新返回 pre-IF 的指令, CPU 就会进入死机状态; 如果我们不想丢弃当前拍返回的指令, 就需要设置一组触发器来保存 pre-IF 级取回的指令, 并且 pre-IF 级暂停发送地址请求。当该组触发器保存有效数据时, 就选择该组触发器保存的数据作为 pre-IF 级取回的指令并送往 IF 级, 在 IF 级 allowin 为 1 后, 该指令进入 IF 级, IF 级不用再等 data_ok, 因为收到了取回的指令。

上述两个问题的解决方案效率较高, 但是复杂度也很高。如果我们为流水线增加一个规则: 仅当 IF 级 allowin 为 1 时 pre-IF 级才对外发出地址请求, 那么当 pre-IF 级 ready_go 为 1 时, IF 级 allowin 一定为 1, 就不会出现“pre-IF-ready_go=1, IF-allowin=0”这种情况, 也就不会出现上述两个复杂的问题。增加该规则后, 取指效率降低了, 但是大大简化了取指状态机的设置。

考虑完 pre-IF 级, 我们来考虑 IF 级的情况。IF 级等待 data_ok 来置 ready_go, 当 ready_go 为 1 且 ID 级 allowin 为 1 时, IF 级的指令流向 ID 级, IF 级维护下一条指令的取指返回或进入无效状态 (IF-valid 为 0)。按 IF-ready_go 和 ID-allowin 的组合情况, 有以下 4 种可能 (以下情况默认 IF-valid 为 1, 表示 IF 级存在有效指令, 如果 IF 级没有有效指令, 自然不会流向 ID 级):

- 1) IF-ready_go=0, ID-allowin=0: 显然 IF 级继续等取指数据返回即可。
- 2) IF-ready_go=0, ID-allowin=1: 显然, 和第 1 种情况一样, IF 继续等待取指数据返回。

3) IF-ready_go=1, ID-allowin=1: 表明 IF 级接收到 data_ok 且正好 ID 级 allowin 为 1, 当前指令就流进 ID 级, 此情况很简单。

4) IF-ready_go=1, ID-allowin=0: 表明 IF 级接收到 addr_ok 但是 ID 级 allowin 为 0, 这种情况最复杂。

对于 IF 级, 上述第 4 种情况无法避免, 并且我们不能丢弃当前拍返回的指令, 因为 IF 级很难重发地址请求, 地址请求是由 pre-IF 级控制的。我们选择的方案是: 设置一组触发器来保存 IF 级取回的指令, 当该组触发器存有有效数据时, 则选择该组触发器保存的数据作为 IF 级取回的指令送往 ID 级, 在 ID 级 allowin 为 1 后, 该指令立即进入 ID 级。此时 IF 级的 ready_go 信号不再只看类 SRAM 总线接口返回的 data_ok, 还要观察来自“保存已返回指令的触发器”的有效信号, 如果该组触发器保存着有效返回值, 那么 IF 级 ready_go 也一定为 1。

3. 考虑例外取消

我们设计的 CPU 支持例外和中断, 那么就存在例外和中断要清空流水线的情况, 记作 Cancel。在引入类 SRAM 总线接口后, Cancel 的情况也需要特别考虑。

我们先来看 pre-IF 级的 Cancel。根据 pre-IF 是否完成了地址请求, 分两种情况讨论。

1) to_fs_valid=0: 表明 pre-IF 级发送的地址请求还未被 CPU 外部接收 (未收到 addr_ok), 依据 8.1.4 节的介绍, 类 SRAM 总线允许请求中途更改请求, 因此直接根据 Cancel 信息调整 pre-IF 级发送的地址请求。Cancel 不会有任何影响。

2) to_fs_valid=1: 表明 pre-IF 级发送的地址请求正好被 CPU 外部接收, 此时存在 Cancel, 要注意 IF 级后续收到的第一个返回的指令数据是对当前被 Cancel 的取指请求的返回。

我们再来看 IF 级的 Cancel。根据 IF 级 allowin 的取值, 可以分两种情况讨论。

1) IF-allowin=1: 表明 IF 级没有有效指令, 或者有有效指令但将要流向 ID 级, 此时 IF-valid 将依据 to_fs_valid 决定置 0 还是 1。此时, 如果 IF 级收到 Cancel, 那么将 IF-valid 时序逻辑置为 0 即可。

2) IF-allowin=0: 表明 IF 级有有效指令, 且该指令无法流向 ID 级, 这时又可以分为两种情况。

- 2-1 IF-ready_go=1: 表明 IF 级正好或已经收到过 data_ok, 但是 ID 级 allowin 为 0, 此时 IF 级没有待完成的类 SRAM 总线事务, 直接 Cancel (将 IF-valid 置为 0) 不会有任何影响。要注意 IF 级有一组保存已返回指令的触发器, Cancel 时要将指示该组触发器保存有指令数据的有效信号也清 0。
- 2-2 IF-ready_go=0: 表明 IF 级正在等待 data_ok, 此时 IF 级有待完成的类 SRAM 总线事务, 可以直接 Cancel (将 IF-valid 置为 0), 要注意 IF 级后续收到的第一个返回的指令数据是对当前被 Cancel 的取指请求的返回。

上述 pre-IF 级第 2 种情况和 IF 级的情况 2-2 有一个共同点: 在 Cancel 后, IF 级后续收到的第一个返回的指令数据是对当前被 Cancel 的取指请求的返回。显然, 后续收到的第一个

返回的指令数据需要被丢弃，不能让其流向 ID 级。我们在设计 IF 级状态机时就要注意该情况，如果不解决这个问题，CPU 中就可能出现如下错误：在例外 Cancel 流水线后，IF 级取回的第一条指令的 PC 和指令码不对应。解决该问题的方法有两个：

1) 在 IF 级状态机引入一个新的状态（该状态表明等一个 data_ok 并丢弃当次返回的指令数据）。

2) IF 级新增一个触发器，复位值为 0。当遇到前述 pre-IF 级第 2 种情况和 IF 级的情况 2-2 时，该触发器置为 1；在收到 data_ok 时，该触发器置为 0。当该触发器为 1 时，组合逻辑将 IF 级的 ready_go 抹成零，从而达到了丢弃第一个返回的指令数据的设计。

以上两种实现方法的本质是一样的。另外，它们都默认了一个前提：在 Cancel 后，IF 级最多只需要丢弃后续返回的一个指令数据。如果在你的设计中，Cancel 后 IF 级最多需要丢弃后续返回的两个指令数据，那就需要对上述实现方法加以改进。

4. 考虑转移延迟槽

在我们的设计中，转移指令在 ID 级生成 br_bus，送往 pre-IF 级参与生成 nextPC。因此，存在这样一种情况：第一拍，转移指令在 IF 级并即将流向 ID 级，此时转移延迟槽指令在 pre-IF 级发送取指地址请求（未收到 addr_ok）；第二拍，转移指令到达 ID 级，生成 br_bus 送往 pre-IF 级，要求更新 nextPC，但是 pre-IF 级依然在维护转移延迟槽指令的取指地址请求发送（未收到 addr_ok），此时 IF 级一定没有有效指令（IF-valid 为 0）。请考虑一下，br_bus 能否更新 nextPC？显然不能，因为一旦更新了，CPU 中就漏掉了转移延迟槽指令。那么应该如何控制 nextPC 不选择 br_bus 送来的信息呢？首先，要识别转移延迟槽指令正处于 pre-IF 级，观察上述第二拍的描述，我们会发现，“ID 级是转移指令，IF-valid 为 0”表明转移延迟槽指令正位于 pre-IF 级，此时 nextPC 只能选择“PC+4”的来源，也就转移延迟槽对应的 PC。

继续考虑第三拍及之后的情况。

1) 如果转移指令继续保持在 ID 级，pre-IF 级收到了 addr_ok，那么转移延迟槽指令就流向了 IF 级，自然采用 br_bus 来更新 nextPC。如果在后续某一拍，转移指令即将离开 ID 级流向 EXE 级，此时转移延迟槽指令在 IF 级，转移目标的指令在 pre-IF 级，pre-IF 级正在使用 br_bus 维护 nextPC，br_bus 上的信息会由于转移指令离开 ID 级而丢失，应该怎么办？解决方案是在 pre-IF 级或 ID 级用一组触发器将 br_bus 的信息暂存起来，这组触发器在转移指令即将离开 ID 级时时序逻辑置为有效，从而负责继续维护 br_bus，以确保转移目标的指令在 pre-IF 级可以继续正确地维护 nextPC。这组触发器什么时候置为无效呢？在转移目标的指令（注意，不是转移延迟槽指令）即将离开 pre-IF 级、流向 IF 级时时序逻辑置为无效。如何识别“转移目标的指令即将离开 pre-IF 级、流水 IF 级”的情况呢？当转移目标的指令在 pre-IF 级，pre-IF-ready_go 为 1 且 IF-allowin 为 1，就是“转移目标的指令即将离开 pre-IF 级、流水 IF 级”的时刻了。因此，关键是识别转移目标的指令正在 pre-IF 级，显然这就是

nextPC 选择 br_bus 的选择信号。

2) 考虑另一种情况。如果转移指令在 ID 并且即将流向 EXE 级, 转移延迟槽指令在 pre-IF 级依然未收到 addr_ok。同样, br_bus 上的信息会因转移指令离开 ID 级而丢失。我们可以采用第 1 种情况类似的处理方法, 在 pre-IF 级或 ID 级用一组触发器将 br_bus 的信息暂存起来。此时, 暂存 br_bus 的触发器有有效信息, 并且 IF-valid 为 0, 表明转移延迟槽指令正位于 pre-IF 级, nextPC 继续选择“PC+4”的来源。当 CPU 继续运行, 转移延迟槽指令进入 IF 级 (IF-valid 为 1), 此时 pre-IF 级是跳转目标的指令, nextPC 选择暂存的 br_bus。又过了几拍, IF 级的转移延迟槽指令已经流入 ID 级, 此时 pre-IF 级的跳转目标的指令未收到 addr_ok, 暂存的 br_bus 依然有效, 此时满足“暂存 br_bus 的触发器有有效信息, IF-valid 为 0”, 却不再表明 pre-IF 级是转移延迟槽指令。因此, 我们要避免这种情况。第一种方法是当暂存 br_bus 的触发器有有效信息且 pre-IF 级 ready_go 为 0 时, 强行让组合逻辑置 IF 级的 ready_go 为 0, 这样 IF 级的 IF-valid 就不会先变成 0, 而是使 IF 级的转移延迟槽指令和 pre-IF 级的转移目标的指令同时流向后续流水级; 第二种方法是在暂存 br_bus 的触发器中再加 1 位 (比如记作 bd_done), 表示转移延迟槽指令刚从 IF 级离开, 在暂存 br_bus 的触发器置为有效时将 bd_done 清 0, 在暂存 br_bus 的触发器有效且 IF-valid 为 1 时将 bd_done 置 1, 当暂存 br_bus 的触发器有效且 bd_done 为 1 时, nextPC 选择暂存的 br_bus 来源。

总结以上内容, 当考虑转移延迟槽时, 我们的设计需要相应做出以下变更:

1) 在 pre-IF 级或 ID 级用一组触发器来暂存 br_bus 的信息。这组触发器在转移指令将要离开 ID 级时, 时序逻辑置为有效, 后续负责继续维护 br_bus; nextPC 选择 br_bus (可能是暂存的 br_bus) 时, 如果 pre-IF-ready_go 为 1 且 IF-allowin 为 1 (表明跳转目标将要流向 IF 级), 将该组触发器时序逻辑置为无效。注意, 这两种情况会同时发生, 但后者优先级更高。也就是说, 当转移指令即将离开 ID 级, 且转移目标指令即将流向 IF 级时, 该组触发器不能置为有效。

2) 当“br_bus (可能是暂存的 br_bus) 有效, 且 IF-valid 为 0”时, 表明位于 pre-IF 级的是转移延迟槽指令, 此时 nextPC 只能选择“PC+4”的来源, 也就转移延迟槽对应的 PC。

3) 对于第 2 点, 我们要排查转移延迟槽指令已经从 IF 级流过的情况, 实现方法有两种: 方法一, 当 br_bus (可能是暂存的 br_bus) 有效, 且 pre-IF 级 ready_go 为 0 时, 强行组合逻辑置 IF 级的 ready_go 为 0; 方法二, 在暂存 br_bus 的触发器中再加 1 位 (比如记作 bd_done), 在暂存 br_bus 的触发器置有效时将 bd_done 清 0, 在暂存 br_bus 有效且 IF-valid 为 1 时, 将 bd_done 置 1, 当 br_bus 有效且 bd_done 为 1 时, nextPC 选择 br_bus 来源。

5. 考虑转移计算未完成的情况

还记得 4.5.2 节的提醒吗? 当转移计算未完成时 (br_bus 里的 br_stall 为 1), 建议 CPU 设计者应该控制指令 RAM 的读使能为 0 (也就是无效)。在未引入总线设计时, 如果没有注意到这一点, 设计的 CPU 不会出错; 但是在后续引入总线设计时, 如果没有注意这一点, 就很可能出错。

如果我们在转移计算未完成时, pre-IF 级用错误的 nextPC 对外发起了取指地址请求, 该请求很有可能会被 CPU 外部接收。CPU 外部接收请求后, 间隔一定的拍数后会向 IF 级返回指令数据, IF 级可能会以为这个数据正好是自己苦苦等待的数据, 它允许这个指令数据流向 ID 级, 这就出错了——IF 级的 PC 和指令码不匹配!

所以, 在转移计算未完成时, pre-IF 级不能在取指端的类 SRAM 接口上置起 req 信号。应该怎么做呢? 就是使用 br_stall 将 req 信号组合逻辑抹成零。

8.2.2 访存设计的考虑

这一节我们来考虑访存设计。和取指相比, 显然访存不需要考虑 8.2.1 节列出的某些情况, 比取指更简单。

我们把访存分为 load 和 store 两类来考虑。load 涉及的逻辑改动与取指的考虑完全一致, 只不过要将取指中的 pre-IF 级和 IF 级换成了 EX 级和 MEM 级。这里还有一个非常小的问题, 提醒大家注意: 如果在你的设计中, MEM 这一级参与前递的有效信号是 MEM 这一级的流水的 valid 信号, 那么务必要把它调整为 MEM 级进入 WB 级的 ms_to_ws_valid。

store 类操作在 EX 级要做的改动与 load 类操作一致。之所以能如此顺利, 要感谢类 SRAM 总线接口中关于 addr_ok 信号的这一段定义: “当操作是写操作时, addr_ok 为 1 表示写地址和写数据均被接收”。store 类操作在 MEM 级需不需要看 data_ok 信号才能进入下一级流水呢? 答案是需要看。因为类 SRAM 总线对于读和写都会返回 data_ok, 如果 store 指令在 MEM 级不看 data_ok 信号就进入 WB 级, 那么后续的 load 指令在 MEM 级看到的 data_ok 信号就有可能是前面 store 对应的 data_ok 信号, 导致 load 指令以为读数据已返回了, 从而获取错误的值。

8.3 AXI 总线协议

关于 AXI 总线协议, 读者可以参考《计算机体系结构基础(第2版)》中 6.2 节的第 1 部分对 AXI 规范中关键性的内容的介绍。在此基础之上, 建议读者学习 AXI v1.0 的规范——“AMBA AXI Protocol Specification v1.0”, 这份规范内容不多, 而且浅显易懂。

我们在接下来的内容中会结合工程实践经验, 谈一谈对于 AXI 总线协议的认识。再次强调, 请大家一定要先结合《计算机体系结构基础(第2版)》或 AXI 规范等资料熟悉 AXI 总线协议, 单独学习本节的内容不足以完成设计。

8.3.1 AXI 总线信号一览

我们将与本章实践任务相关的 AXI 总线信号列举在表 8-4 中。其中标为黑体的信号是关键信号, 大家必须要掌握。备注栏中是我们针对本章实践任务给出的一些设计建议。例如, 信号 arlen 的备注是“固定为 0”, 表示建议读者在设计实现的过程中, 可以将这个信号的输

出恒置为 0。因为目前我们没有实现 Cache，所以任何读请求都只需要一次总线传输就能完成，相应的 arlen 就为 0。信号 rresp 的备注是“可忽略”，意味着你们设计的接口中可以完全不关注 rresp 这个信号，因为目前我们不考虑 Bus Error 情况下的特殊处理，也不会利用总线进行原子访问。

表 8-4 32 位 AXI 接口信号一览

信号	位宽	方向	功能	备注
AXI 时钟与复位信号				
aclk	1	input	AXI 时钟	
aresetn	1	input	AXI 复位，低电平有效	
读请求通道，(以 ar 开头)				
arid	[3:0]	master → slave	读请求的 ID 号	取指置为 0； 取数置为 1
araddr	[31:0]	master → slave	读请求的地址	
arlen	[7:0]	master → slave	读请求控制信号，请求传输的长度（数据传输拍数）	固定为 0
arsize	[2:0]	master → slave	读请求控制信号，请求传输的大小（数据传输每拍的字节数）	
arburst	[1:0]	master → slave	读请求控制信号，传输类型	固定为 0b01
arlock	[1:0]	master → slave	读请求控制信号，原子锁	固定为 0
arcache	[3:0]	master → slave	读请求控制信号，Cache 属性	固定为 0
arprot	[2:0]	master → slave	读请求控制信号，保护属性	固定为 0
arvalid	1	master → slave	读请求地址握手信号，读请求地址有效	
arready	1	slave → master	读请求地址握手信号，slave 端准备好接收地址传输	
读响应通道，(以 r 开头)				
rid	[3:0]	slave → master	读请求的 ID 号，同一请求的 rid 应和 arid 一致	0 对应取指； 1 对应数据
rdata	[31:0]	slave → master	读请求的读回数据	
rresp	[1:0]	slave → master	读请求控制信号，本次读请求是否成功完成	可忽略
rlast	1	slave → master	读请求控制信号，本次读请求的最后一拍数据的指示信号	可忽略
rvalid	1	slave → master	读请求数据握手信号，读请求数据有效	
rready	1	master → slave	读请求数据握手信号，master 端准备好接收数据传输	
写请求通道，(以 aw 开头)				
awid	[3:0]	master → slave	写请求的 ID 号	固定为 1
awaddr	[31:0]	master → slave	写请求的地址	
awlen	[7:0]	master → slave	写请求控制信号，请求传输的长度（数据传输拍数）	固定为 0
awsize	[2:0]	master → slave	写请求控制信号，请求传输的大小（数据传输每拍的字节数）	
awburst	[1:0]	master → slave	写请求控制信号，传输类型	固定为 0b01

(续)

信号	位宽	方向	功能	备注
awlock	[1:0]	master → slave	写请求控制信号, 原子锁	固定为 0
awcache	[3:0]	master → slave	写请求控制信号, Cache 属性	固定为 0
awprot	[2:0]	master → slave	写请求控制信号, 保护属性	固定为 0
awvalid	1	master → slave	写请求地址握手信号, 写请求地址有效	
awready	1	slave → master	写请求地址握手信号, slave 端准备好接收地址传输	
写数据通道, (以 w 开头)				
wid	[3:0]	master → slave	写请求的 ID 号	固定为 1
wdata	[31:0]	master → slave	写请求的写数据	
wstrb	[3:0]	master → slave	写请求控制信号, 字节选通位	
wlast	1	master → slave	写请求控制信号, 本次写请求的最后一拍数据的指示信号	固定为 1
wvalid	1	master → slave	写请求数据握手信号, 写请求数据有效	
wready	1	slave → master	写请求数据握手信号, slave 端准备好接收数据传输	
写响应通道, (以 b 开头)				
bid	[3:0]	slave → master	写请求的 ID 号, 同一请求的 bid、wid 和 awid 应一致	可忽略
bresp	[1:0]	slave → master	写请求控制信号, 本次写请求是否成功完成	可忽略
bvalid	1	slave → master	写请求响应握手信号, 写请求响应有效	
bready	1	master → slave	写请求响应握手信号, master 端准备好接收写响应	

8.3.2 理解 AXI 总线协议

《计算机体系结构基础(第2版)》和 AXI 规范已经用严谨、正确、全面的语言对 AXI 协议是什么进行了介绍, 我们在本节中不准备重复介绍这些概念、定义。我们将基于实际的工程实践, 根据我们的理解来解释一下 AXI 协议中各种信号、规定的设计意图。由于我们并不是 AXI 协议规范的设计者, 所以接下来的解释难免有狭隘之处。这些内容只是帮助读者加深理解的参考。

1. 握手

前面说过, 总线是处理器和内存、外设交互的通道。既然要进行交互, 双方就要步调一致。要想做到步调一致, 可以采用两种方式: 事先约定时间, 或者通过握手。AXI 总线协议采用的是握手机制。

我们之前设计的 CPU 流水线之间也采用了握手机制, 所以大家应该知道完成一次握手需要一对信号——一个请求, 一个应答。AXI 协议中各个通道上都有 valid 和 ready 两个信号, 它们就是用来实现握手的。因为 AXI 这套协议是面向一个同步的数字逻辑电路设计来定义的, 所以 valid 和 ready 两个信号间不是异步的互锁。主、从双方都是在时钟上升沿看这一对信号。图 8-5 是我们从 AXI 规范中摘录的表述 valid 和 ready 关系的时序示意图。图中箭

头所指的那个上升沿是传输发生的时刻。

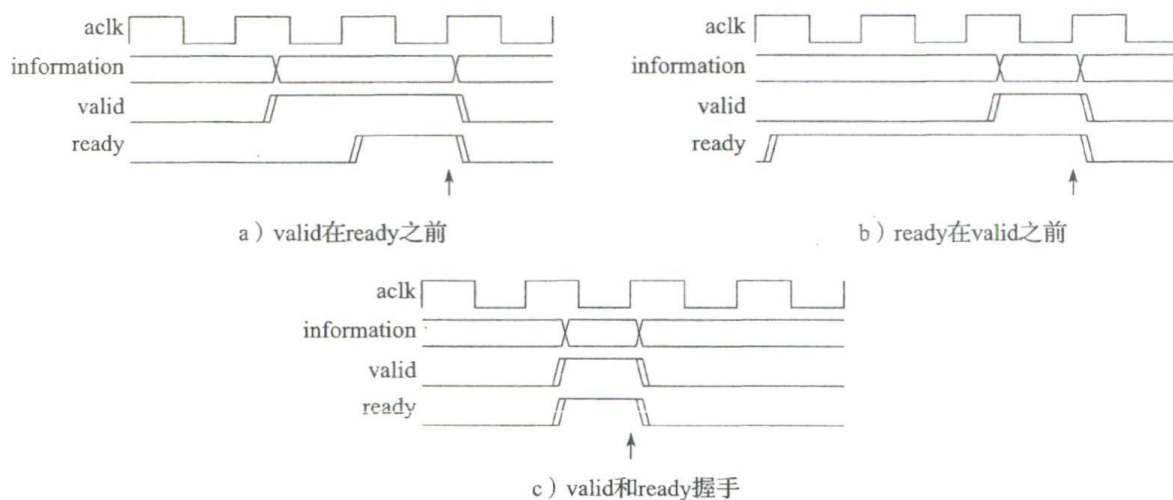


图 8-5 AXI 总线协议中 valid 和 ready 的关系

每个通道上发起方和接收方之间的握手机制和我们 CPU 内部流水线间的握手机制是完全一样的。你可以想象每个通道上都有两级流水级缓存，发起方（valid 信号是 output 的一方）那里有一级，接收方（ready 信号是 output 的一方）那里有一级。一次总线请求的握手交互就是将发起方的流水级缓存中的信息写到接收方的流水级缓存中。AXI 总线里的 valid 就是我们 CPU 里面的 p1_to_p2_valid，AXI 总线里的 ready 就是我们 CPU 里面的 p2_allowin。

图 8-5 中其实还包含着一个重要的信息：在 AXI 总线协议中，valid 和 ready 这对握手信号置为有效时没有先后关系。事实上，AXI 规范中还明确了每个通道中 valid 和 ready 的依赖关系：

- valid 的置有效一定不能依赖于 ready 是否有效。
- ready 的置有效可以依赖于 valid 是否有效。

上述严格限制是为了避免死锁。因此提醒大家，一定不要根据 ready 是否为 1 来决定如何置 valid。

2. 总线事务和总线传输

主、从双方通过总线进行的读写操作是一个交互的过程，这个过程可能涉及很多信号并且在总线上持续多个周期。对于高性能的片上总线，如 AXI 总线，同一时刻总线上可能进行着多个不同的读、写操作。因此，仅从信号的角度不太容易说清楚总线的行为，于是就有了总线事务（Transaction）这个概念。一次总线事务对应于一次完整的读或者写的过程。它是一个描述总线行为的更高层次的抽象，也是大家设计总线接口时的主要着眼点。

一个总线事务包含多个总线传输（Transfer）。总线传输只针对 valid 和 ready 同时有效（即握手成功）的那个时钟周期。图 8-6 中给出了一个 AXI 的读总线事务，包括读请求通道上的一次传输（虚线方框）和读响应通道上的四次传输（实线方框）。

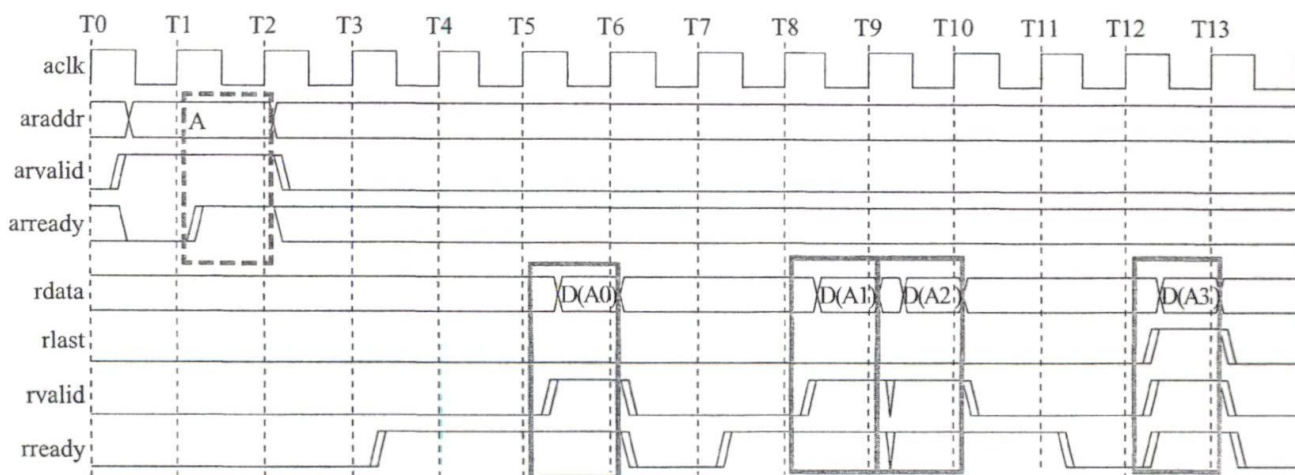


图 8-6 AXI 总线事务与总线传输

《计算机体系结构基础 (第2版)》一书 6.2 节的第 1 部分和 AXI 规范给出了 AXI 总线单次读、重叠读、单次写的总线事务的时序图, 请读者可参考这些资料进一步学习和理解 AXI 总线。

3. 地址、大小、数据

主、从双方进行交互时, 交互的是数据。这就涉及下面一系列问题:

- 如何区分这些数据呢? 通过地址来区分。
- 寻址的粒度最小到什么度? AXI 总线的寻址粒度是字节。
- 每次交互的数据量可能有多有少, 怎么办? 主方会告诉从方传输数据量的大小。

可见, 对于任何总线协议, 地址、数据、大小 (有时会进一步分成传输次数和传输宽度两个要素) 这些要素是必不可少的^①。对应到 AXI 总线协议上就是 araddr、arsize、arlen、rdata、awaddr、awsize、awlen、wdata、wstrb 这些信号。这些信号是主要信号, 应该熟练掌握。

4. 多个通道

通道对应底层的信号线。通道就是马路, 前面说的地址、数据信息是马路上行驶的汽车。AXI 协议中定义了 5 个通道, 目的是实现非常高的总线传输性能。

我们先来说读和写分开。因为读和写操作都要交互地址和数据, 所以读写通道合在一起的话, 读的时候就不能写, 写的时候就不能读; 如果给读写分配各自的通道, 两个操作就可以各自进行, 互不干扰。无论是合并还是分开, 本质上是在资源和性能之间进行权衡, 取决于设计者的关注点。AXI 关注的是性能, 所以采取将读和写通道分开的设计。感兴趣的读者可以看一下 AHB 总线协议, 这个协议采取的就是将读、写合在一起的方案。

接下来再说说读或者写各自又划分出来的几个通道。对于读请求通道和读响应通道的作用, 我们可以用一个比喻来说明。读请求通道相当于买家在购物网站上下订单 (注意这个过程是有握手的), 读数据通道相当于快递公司把货物从卖家送到买家手上 (显然这个过程也

① 有些总线是双向传输的, 所以还有方向这个要素。不过 AXI 总线是单向的, 故不存在这个要素。

是有握手的)。写请求通道、写响应通道和读通道类似，只不过可以比喻成买家向卖家退货，买家和卖家协商好需要退货（写请求通道），快递公司把货物从买家运到卖家（写数据通道）。卖家收到退回的货物后，会给买家发回确认的消息，这就是写响应通道。

5. 多通道间的事务握手依赖关系

因为一次读事务和一次写事务要在多个通道上通过多次传输完成，而每个通道都有自己的握手机制，那么围绕同一次事务，不同通道间的握手要有遵循什么依赖关系呢？这也是需要定义的。图 8-7 是我们从 AXI 规范^①中摘录的关于握手依赖关系的示意图。图中的双箭头表示必须存在依赖关系，单箭头表示可以存在依赖关系。我们要重点关注双箭头所表示的必须存在的依赖关系。

在图 8-7 中，上半部表示一次读事务，只有在 arvalid 和 arready 都有效，从方才能将返回数据对应的 rvalid 置为有效。这是符合直觉的。大家要记住的是，如果一个读事务的读请求还没有被从方接收，那么这个时候 rvalid 和当前的读事务没有任何关系，要确保你设计的状态机不要错误动作。

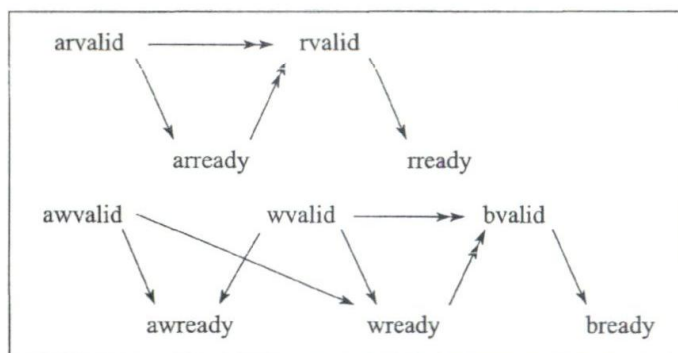


图 8-7 AXI 总线事务握手依赖关系

在图 8-7 的下半部，只有写请求和写数据的最后一次传输都被从方接收以后，从方才会反馈写响应。这也是符合直觉的。与上面读事务的处理一样，只有等到写请求和写数据都发出之后，才有看 bvalid 的必要。这个部分有一个有意思的地方是，写请求和写数据之间没有任何依赖关系，所以从理论上讲，可以先发送写数据再发送写请求。不过，对于 CPU 总线接口设计来说，这样做没有任何好处，反而违背直觉。建议大家将 awvalid 和第一次传输的 wvalid 一并置为有效。

6. 并发访问

对于买家来说，你可以先下单购买一个商品，收货之后再购买下一个商品，也可以同时下多个订单，然后等着收货。对于卖家来说，可以一次只处理一个订单，也可以同时处理多个订单。对应到 AXI 总线上，后者就是并发访问。对于某个主方来说，它可以连续发出多个请求，即使先前发出的请求所需要的数据还没有传输完毕。对于某个从方来说，它可以在没有传输完之前请求所需要的数据的时候，就接收来自一个或多个主方发来的新请求。这种并发访问得以实现的客观保证是请求通道和数据通道是分离的。所以，在 AXI 总线协议中，读通道划分为读请求和读响应通道，写通道划分为写请求、写数据和写响应通道。

7. 乱序响应

通俗地说，事务的乱序响应就是后发出的请求可能先返回数据。允许事务的乱序响应是

^① 写事务的依赖关系图是从 AXI3 规范中摘录的，因为我们觉得这张图表述更加合理。

为了提升总线的传输性能。当系统中集成了多个主方，或者主方支持乱序执行机制，那么总线乱序响应会显著提升性能。本书实践任务只是设计了一个单核的静态流水线 CPU，通过总线的乱序响应获得的性能提升很少。

我们在设计 CPU 总线接口的时候要关注乱序可能带来的影响，保证设计正确。对于初学者来说，最容易在读通道上面出问题。请注意，除非发出的一连串读请求都是采用同一个 ID，否则就要考虑后发的请求的数据先返回的情况。当你希望先发出的请求一定先返回，那么就必须将这些请求的 ID 置为相同值。

8. ID

因为 AXI 总线协议支持并发访问和乱序响应，所以请求和响应之间的对应关系需要额外的信息来维护。具体来说，这是通过各个通道上的 ID 信号来完成的。还记得我们之前所说的“总线事务”的视角吗？AXI 上的 ID 是事务的 ID。举例来说，主方发出一个读请求，arid 设为 0，过了一段时间，主方接收到一个读响应，它如何判断返回的数据就是自己之前发出的请求所需要的呢？答案是看 rid 是不是等于 0。写的处理方式也是类似的，写地址是从写请求通道发出去的，写数据是从写数据通道发出去的，从方如何知道一个写数据对应哪个写地址？它会看写地址对应的 awid 和写数据的 wid 是不是相等。

不过，千万要记住，AXI 协议规范中规定：不同的事务可以设置不同的 ID。换言之，不同的事务也可以设置相同的 ID。

9. 写响应通道的作用

很多初学者不理解为什么 AXI 协议中要定义写响应通道，感觉它好像没有用。要搞清楚这个问题，我们必须站在片上系统的视角去观察、分析 AXI 总线处理“写后读相关”访问时的行为。例如，假设系统中只有一个主设备是 CPU，地址 A 处的原值是 0，现在 CPU 通过 AXI 总线向 A 地址处写 1，在最后一个数据写出去（wvalid && wlast && wready == 1）之后，CPU 再通过 AXI 总线读 A 地址，请问会返回什么值？你是不是会认为它显然应该是 1？但是，在 AXI 总线下，返回值是 0 也合规。所谓合规，就是指总线的各个地方都满足 AXI 协议规范，没有实现错误，也有可能返回 0。

这种违背直觉的现象是怎么出现的呢？这是两个原因共同作用的结果。原因一是读、写通道分离。分离就意味着这两个通道彼此间没有相互作用，大家各干各的。原因二是 AXI 总线从主方到最终访问的从方之间可以有任意多级缓存。在这两个原因的共同作用之下，完全有可能出现主方先发完的写地址和写数据被堵在通道上，后发出的读请求从读通道畅通无阻地传输下去，最终访问的从方先看到了读请求后看到写请求，因此从方返回一个旧值 0。

这种情况看起来非常特殊，但它是合理的，这意味着它出现的概率不是 0。设计一台计算机时，我们必须防范这些特殊的可能导致出错的情况。

如何处理这种情况呢？看来即使写数据请求发出去了，读请求也不见得能立即发出去。要等多久呢？等一万年够不够？不够，因为有可能写数据请求要被堵两万年。注意，是有可

能，只要不能肯定这种情况出现的概率是 0，那就有可能出现。最终的解决方案是在 AXI 协议中引入写响应通道。这个消息是从方发出的，表明它已经接收到了写数据，当主方接收到这个消息之后再发出读请求，就一定是安全的。因为电信号的传播速度不会超过光速，所以读请求到达从方的时间一定在写数据到达从方之后。

10. 突发传输模式

为什么要设计突发传输模式？我们通过例子来说明。假设总线上读数据的宽度是 32 比特，那么当你想读 512 比特地址连续的数据时，该怎么发送总线请求呢？一种方式是，发送 16 次 4 字节的读地址请求，这意味着要在读地址通道进行 16 次握手。如果因为时序的原因，主、从方之间的握手不可能逐拍连续完成，那么这 16 次握手就会耗费很多时间。于是我们想到是不是可以设计一种模式，使得只与从方交互一次，就能把传送 512 比特地址连续数据的事情交代完毕？这就是突发传输模式。为了描述清楚一个突发传输模式，需要起始地址、地址变化规律、地址变化次数三个信息。以读通道为例，这分别对应 `araddr`、`arburst`、`arlen`。目前，我们的设计中还没有实现 Cache，在 32 位数据宽的 AXI 总线接口上不会有突发传输需求。所以，可以等到实现 Cache 的时候再调整总线接口的设计。

8.3.3 类 SRAM 总线接口信号与 AXI 总线接口信号的关系

如果将类 SRAM 总线接口信号与 AXI 总线接口信号进行对比，可以发现类 SRAM 总线中将读写合并在一起，但还是将请求和响应分离开来，不同总线事务的请求和响应可以重叠在一起，从而提高总线的传输效率。类 SRAM 总线不支持响应的乱序返回，从而简化了设计。总体来看，类 SRAM 总线接口比 AXI 总线接口的行为简单，所以将类 SRAM 总线转换成 AXI 总线是简单的，因为大多数信号都能找与其对应的信号。

对于 `req` 信号来说，当 `req=1` 且 `wr=0` 时，对应 AXI 总线的 `arvalid` 信号；当 `req=1` 且 `wr=1` 时，对应 AXI 总线的 `awvalid` 信号和 `wvalid` 信号。

对于 `addr_ok` 信号来说，当对应的是读事务的时候，其对应 AXI 总线的 `aready`。当对应的是写事务的时候，没有简单的对应信号，它的含义其实是 AXI 总线上的 `awready` 和 `wready` 都已经或正在为 1。此处的对应关系有些复杂，读者在设计“类 SRAM-AXI”转接桥时需要关注。

对于 `data_ok` 信号来说，当对应的是读事务的时候，其对应 AXI 总线的 `rvalid`；当对应的是写事务的时候，对应 AXI 总线的 `bvalid`。

对于 `addr` 信号来说，当对应的是读事务的时候，其对应 AXI 总线的 `araddr`；当对应的是写事务的时候，其对应 AXI 总线的 `awaddr`。

对于 `size` 信号来说，当对应的是读事务的时候，其对应 AXI 总线的 `arsize`。当对应的是写事务的时候，其对应 AXI 总线的 `awsize`。

此外，`rdata` 对应 AXI 总线的 `rdata`，`wdata` 对应 AXI 总线的 `wdata`，`wstrb` 对应 AXI 总线的 `wstrb`。

8.4 类 SRAM-AXI 的转接桥设计

理解了 AXI 总线协议和类 SRAM 总线协议之后，我们开始考虑如何设计一个类 SRAM-AXI 的转接桥。

8.4.1 转接桥的顶层接口

目前我们设计的 CPU 有一个指令段的类 SRAM 接口和一个数据段的类 SRAM 接口。我们设计的转接桥是供 CPU 使用的，所以它要有两个类 SRAM 接口。作为一个简单的 CPU，对外通常有一个总线接口，所以我们设计的转接桥要有一个 AXI 接口。整个转接桥与 CPU 其余部分，以及与 SoC 中的其他部分的关系如图 8-8 所示。

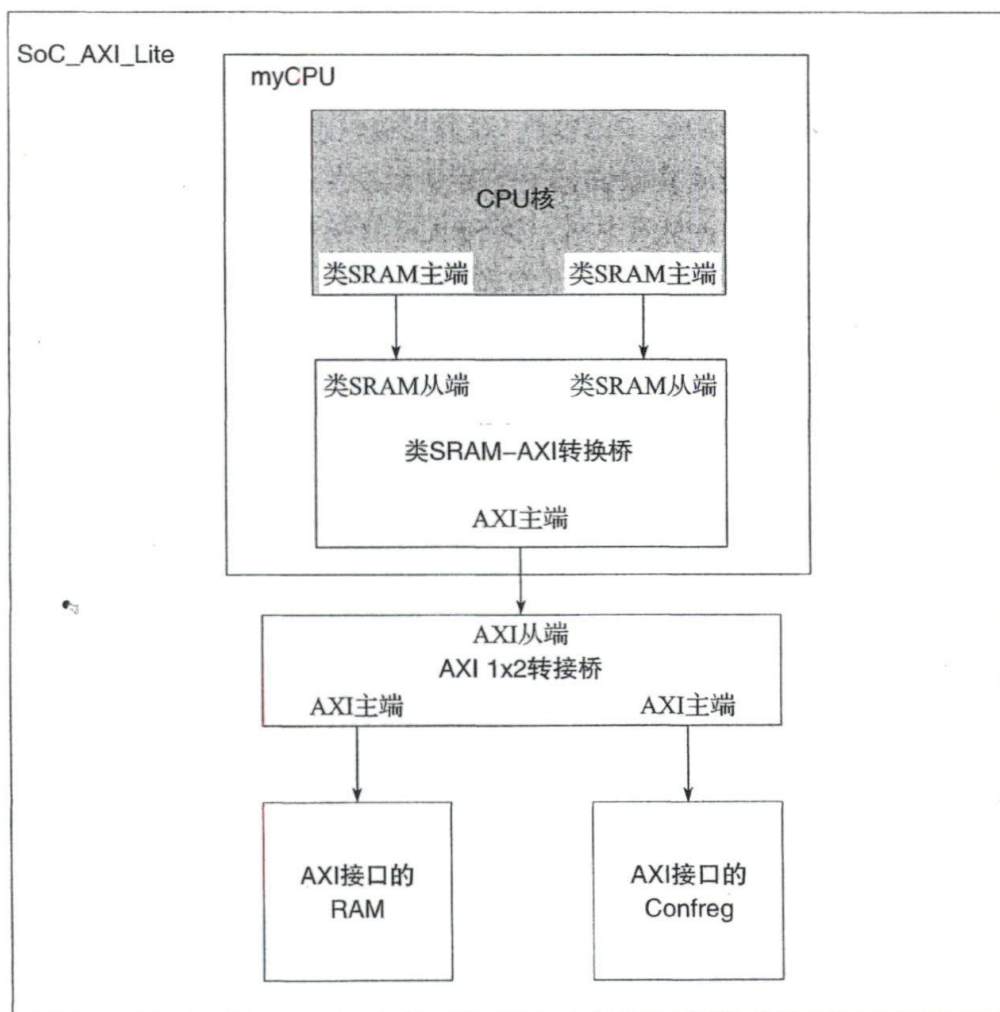


图 8-8 SoC_AXI_Lite 中的类 SRAM-AXI 转接桥

需要提醒的是，类 SRAM-AXI 转接桥中的类 SRAM 是从端而不是主端，千万不要把这两个接口上的信号方向弄反。写完代码后，最好仔细人工检查一遍端口定义的方向，避免陷入“仿真通过，上板不过”的困境中。

8.4.2 转接桥的设计要求

本节会给出转接桥的设计要求。其中，有的要求是为了保证功能正确，有的要求是为了与我们已有的设计和验证环境匹配。这些要求必须在设计中予以贯彻。

1) aresetn 有效期间，AXI Master 端的所有 valid 类输出必须为 0，所有 ready 类输出不能为 X 值。

2) AXI Master 端的所有 valid 输出信号的置 1 逻辑中，一定不能允许组合逻辑来自同一通道的 ready 输入信号（时序逻辑来自 ready 是可以的）。

3) 对于 AXI Master 端的所有 ready 输出信号，一定不能允许组合逻辑来自同一通道的 valid 输入信号（时序逻辑来自 valid 是可以的）。

4) 无论读、写请求，类 SRAM 接口上的事务要和 AXI 接口上的事务严格一一对应。特别要注意，既不要将类 SRAM 接口上的一个事务在 AXI 总线上重复发送多次，也不要将类 SRAM 发来的多个地址连续的读事务或写事务合并成一个 AXI 总线事务。

5) 在 AXI Master 端的读请求、写请求、写数据通道上，如果 Master 输出的 valid 置为 1 的时候对应的 ready 是 0，那么在 ready 信号变为 1 之前，不允许 Master 变更该通道的所有输出信号。这一点与类 SRAM 总线不同，类 SRAM 总线允许中途更改请求。

6) AXI Master 端在发起读请求时，要先确保该请求与“已发出请求但尚未接收到写响应的写请求”不存在地址相关。最简单直接的解决方式是，只要有写请求，就停止发起读请求直至 Master 端收到写响应。更精细、高效的处理方式是记录那些“已发出请求但尚未接收到写响应的写请求”的地址、位宽、字节写使能等信息，后续读请求查询并比较这些信息后再决定是否发出。

7) 在转接桥内部不允许做写到读的数据前递，如有写后读相关，一定通过阻塞读来处理。

8.4.3 转接桥的设计建议

本节将给出转接桥的设计建议，所有建议的出发点是牺牲一些性能和面积来换取控制逻辑的简洁性。既然是建议，意味着你可以采纳也可以不采纳。不采纳的好处是，你可以通过设计阶段更多的纠结和调试阶段的 bug 修正来深入理解为什么要提出这些建议。

1) 除了可以置为常值的信号外，所有 AXI Master 端的输出直接来自触发器 Q 端。

2) 为读数据预留缓存，从 rdata 端口上收到的数据先保存到这个预留缓存中。

3) AXI 上最多支持几个“已完成读请求握手但数据尚未返回的读事务”，就预留同样数目的 rdata 缓存。

4) 取指对应的 arid 恒为 0，load 对应的 arid 恒为 1。

5) 控制 AXI 读写的状态机分为独立的 4 个状态机：读请求通道一个，读响应通道一个，写请求和写数据共用一个，写响应一个。

6) 类 SRAM Slave 端接的输入信号不用锁存后再使用。

7) 数据端发来的类 SRAM 总线的读请求优先级固定高于取指端发来的类 SRAM 总线的读请求。

8) 类 SRAM Slave 端输出的 `addr_ok` 和 `data_ok` 信号若是来自组合逻辑, 那么这个组合逻辑中不要引入 AXI 接口上的 `valid` 和 `ready` 信号。

8.5 任务与实践

完成本章的学习后, 读者应能够完成以下 3 个实践任务:

- 1) 添加类 SRAM 总线支持, 参见 8.5.1 节, 实践资源见 lab10.zip。
- 2) 添加 AXI 总线支持, 参见 8.5.2 节, 实践资源见 lab11.zip。
- 3) 完成 AXI 随机延迟验证, 参见 8.5.3 节, 实践资源复用 lab11.zip。

为完成以上实践任务, 需要参考的文档包括但不限于:

- 1) 本章内容。
- 2) AMBA AXI 总线协议官方文档。

8.5.1 实践任务一: 添加类 SRAM 总线支持

本实践任务有以下两个要求:

- 1) 将 CPU 顶层接口修改为类 SRAM 总线接口。
- 2) 在 CPU_CDE_SRAM 环境里完成随机延迟的 `func_lab9` 功能验证。

本实践任务的环境不再沿用 lab3 ~ lab9 的 CPU_CDE, 而是使用类 SRAM 接口的 CPU 设计实验开发环境 CPU_CDE_SRAM, 见 lab10.zip。CPU_CDE_SRAM 环境的使用方法与 CPU_CDE 环境基本一致, 目录结构稍有不同, 如下所示。

<code> -cpu132_gettrace/</code>	该目录与 CPU_CDE 相同
<code> -soft/</code>	该目录与 CPU_CDE 相同
<code> -mycpu_sram_verify/</code>	读者实现的类 SRAM 总线 CPU 的验证环境, 与 CPU_CDE 大部分类似
<code> --rtl/</code>	SoC_lite 设计代码目录
<code> --soc_sram_lite_top.v</code>	SoC_lite 的顶层文件
<code> --myCPU/*</code>	自己实现的 CPU 的 RTL 代码
<code> --CONFREG/</code>	confreg 模块, 用于访问 CPU 与开发板上数码管、拨码开关等外设
<code> --BRIDGE/</code>	1x2 的桥接模块, CPU 的 data_sram 接口同时访问 confreg 和 data_ram
<code> --ram_wrap/</code>	以类 SRAM 接口封装的 RAM 模块
<code> --xilinx_ip/</code>	定制的 Xilinx IP, 包含 clk_pll、inst_ram、data_ram
<code> --testbench/</code>	功能仿真验证平台
<code> --mycpu_tb.v</code>	功能仿真顶层, 该模块会抓取 debug 信息与 golden_trace.txt 进行比较
<code> --run_vivado/</code>	Vivado 工程的运行目录
<code> --soc_lite.xdc</code>	Vivado 工程设计的约束文件
<code> --mycpu_prj1/</code>	创建的第一个 Vivado 工程, 名字为 mycpu_prj1
<code> --*.xpr</code>	Vivado 创建的工程文件, 可直接打开
<code> --*</code>	创建的其他 Vivado 工程

请参考下列步骤完成本实践任务：

- 1) 学习 8.1 节、8.2 节的内容，重点理解类 SRAM 总线协议。
- 2) 将 lab10.zip 解压到路径上无中文字符的目录里，环境为 CPU_CDE_SRAM。
- 3) 解压 lab9 提供的 lab9.zip，将 func_lab9/ 拷贝到 CPU_CDE_SRAM/soft/ 目录里。
- 4) 打开 cpu132_gettrace 工程（CPU_CDE_SRAM/cpu132_gettrace/run_vivado/cpu132_gettrace/cpu132_gettrace.xpr）。
- 5) 重新定制 cpu132_gettrace 工程中的 inst_ram，此时选择加载 func_lab9 的 coe（CPU_CDE_SRAM/soft/func_lab9/obj/inst_ram.coe）。
- 6) 运行 cpu132_gettrace 工程的仿真（进入仿真界面后，直接点击 run all 等待仿真运行完成），生成新的参考 Trace 文件 golden_trace.txt（CPU_CDE_SRAM/cpu132_gettrace/golden_trace.txt）。注意，要等仿真运行完成后，golden_trace.txt 才有完整的内容。
- 7) 编写 RTL 代码，将 lab9 完成的 myCPU 顶层接口调整为类 SRAM 总线接口。
- 8) 将新完成的 myCPU 代码放在 CPU_CDE_SRAM/mycpu_sram_verify/rtl/myCPU/ 目录里。
- 9) 打开 myCPU 工程（CPU_CDE_SRAM/mycpu_sram_verify/run_vivado/mycpu_prj1/mycpu_prj1.xpr）。
- 10) 通过“Add Sources”将新的 myCPU 源码添加到工程中。
- 11) 对 myCPU 工程中的 axi_ram 进行重新定制，此时选择加载 func_lab9 的 coe（CPU_CDE_SRAM/soft/func_lab9/obj/inst_ram.coe）。
- 12) 运行 myCPU 工程的仿真（进入仿真界面后，直接点击 run all），开始调试。
- 13) 仿真通过后，综合实现后生成比特流文件，进行上板验证（如果没有实验箱，请跳过这个步骤）。

1. 上板验证的要求

上板验证时，要求“随意切换拨码开关后按复位键”，CPU 能通过 func_lab9 里 94 个功能点的验证。

“随意切换拨码开关后按复位键”是为了设定初始的随机种子（随机种子用于生成 inst/data ram 访问时的随机延迟拍数），开始运行功能验证。由于功能验证程序里的指令较多，随机种子不同会导致取指或访存的随机延迟拍数不同，CPU 执行状态也会大不相同，所以切换初始随机种子后出现错误是很常见的。

请注意上板验证的具体操作：将 8 个拨码开关切换为随意状态，按复位键；松开复位键后，数码管开始累加，此时可以切换开关来控制 wait_1s 的累加速度。

2. 拨码开关的功能

从上面的内容可以看出，上板验证时，拨码开关有两个功能：

- 功能一：复位期间，拨码开关控制初始随机种子，进而控制 CPU 取指或访存的随机延迟拍数的生成序列。

- 功能二：复位后，拨码开关控制 wait_1s 的循环次数，也就是控制数码管累加的速度。

对于功能二，在 94 个功能点测试中，每两个功能点之间会穿插一个 wait_1s 函数，wait_1s 通过一段循环完成计时的功能：wait_1s 的循环次数由拨码开关控制，可设置循环次数为 $(0 \sim 0xaaaa) * 2^9$ 。上板验证时，建议在复位后，通过拨码开关选择合理的 wait_1s 延时。

对于功能一，为尽可能验证 myCPU 的功能，CPU_CDE_SRAM 里对访存延迟设定了一个随机机制：访存延迟拍数是通过一个 32 位的伪随机数发生器（在 confreg.v 文件里实现）生成的。在仿真验证时，该伪随机数发生器初始随机种子由 confreg.v 里的宏 RANDOM_SEED 指定；在上板验证时，其初始随机种子由复位期间采样到的拨码开关状态来指定。

实验箱上共有 8 个拨码开关，实际电平是：拨上为 0，拨下为 1。但是为了便于以下的描述，我们记作：拨上为 1，拨下为 0。16 个 LED 单色灯的实际电平是：驱动 0 亮，驱动 1 灭。同样，我们记作：驱动 1 亮，驱动 0 灭。

上板验证时，按下复位键，会自动采样 8 个拨码开关的值作为初始随机种子，且会显示初始随机种子低 16 位到单色 LED 灯上。上板时随机种子与拨码开关的对应关系如表 8-5 所示，需要注意的是，延迟类型依据拨码开关的值分为三类：长延迟、短延迟和无延迟类型。在上板验证时，应当覆盖这三类延迟。

表 8-5 上板验证时初始随机种子设定

约定，8 个拨码开关：拨上为 1，拨下为 0，记作 switch[7:0]		
约定，16 个单色 LED 灯：驱动 1 亮，驱动 0 灭，记作 led[15:0]		
对应关系：led[15:0]={2{switch[7]}}, {2{switch[6]}}, {2{switch[5]}}, {2{switch[4]}}, {2{switch[3]}}, {2{switch[2]}}, {2{switch[1]}}, {2{switch[0]}}		
随机延迟类型分为 3 种类型：		
1) 长延迟类型：随机种子低 8 位不为 8'hff，即 seed_init[7:0]!=8'hff		
2) 短延迟类型：随机种子低 8 位为 8'hff，即 seed_init[7:0]==8'hff（排除无延迟类型）		
3) 无延迟类型：随机种子低 16 位为 16'h00ff，即 seed_init[15:0]==16'h00ff		
拨码开关状态	LED 灯显示	实际初始种子 seed_init
8'h00	16'h0000	{7'b1010101, 16'h0000}
8'h01	16'h0003	{7'b1010101, 16'h0003}
8'h02	16'h000c	{7'b1010101, 16'h000c}
8'h03	16'h000f	{7'b1010101, 16'h000f}
...	...	...
8'hff	16'hffff	{7'b1010101, 16'hffff}

3. “仿真通过，上板不过”的调试方法

【情况一】上板验证时发现数码管没有任何累加。

这可能是由于以下问题之一导致的：

- 1) 多驱动。

- 2) 模块的 input/output 端口接入的信号方向不对。
- 3) 时钟复位信号接错。
- 4) 代码不规范, 阻塞赋值乱用, always 语句随意使用。
- 5) 仿真时控制信号有“X”。仿真时, 有“X”调“X”, 有“Z”调“Z”。特别要注意的是, 设计的顶层接口上不要出现“X”和“Z”。
- 6) 时序违约。
- 7) 模块里的控制路径上的信号未进行复位。

【情况二】上板验证时发现在某些随机种子下测试通过, 在另一些随机种子情况下出错。请按以下步骤进行调试:

1) 确认上板验证时出错的初始随机种子, 修改 CPU_CDE_SRAM/rtl/CONFREG/confreg.v 里的宏 RANDOM_SEED 的定义值, 改为出错时的初始随机种子, 随后进行仿真。如果有错, 则进行调试; 如果发现仿真没有出错, 则在上板验证时寻找下一个出错的初始随机种子, 同样设定好 RANDOM_SEED 后进行仿真, 如果尝试多个初始随机种子后仿真都没有出错则转到步骤 2。

2) 当遇到“相同初始随机种子, 仿真无法复现上板的错误”的情况时, 请采取以下措施: 排查列出的可能原因, 复查代码和反思设计, 也可以使用 Vivado 的逻辑分析仪进行在线调试, 参考附录 D 的第 1 节。

【情况三】上板验证时发现任意随机种子下, 都只有部分功能点测试通过。

这时可能以上两种情况的原因都存在, 请依次排查。

8.5.2 实践任务二: 添加 AXI 总线支持

本实践任务的要求如下:

1) 将 CPU 顶层接口修改为 AXI 总线接口。CPU 对外只有一个 AXI 接口, 需在内部完成取指和数据访问的仲裁。推荐在本任务中实现一个类 SRAM-AXI 的 2x1 的转接桥, 然后拼接上实践任务一完成的类 SRAM 接口的 CPU, 将 myCPU 封装为 AXI 接口。

2) 在 CPU_CDE_AXI 环境里完成固定延迟的 func_lab9 功能验证。

本实践任务的环境既不是 lab3 ~ lab9 的 CPU_CDE, 也不是 lab10 的 CPU_CDE_SRAM, 而是使用 AXI 接口的 CPU 设计实验开发环境 CPU_CDE_AXI, 见 lab11.zip。CPU_CDE_AXI 环境的使用方法与 CPU_CDE 环境基本一致, 目录结构稍有不同, 如下所示。

-cpu132_gettrace/	该目录与 CPU_CDE 相同
-soft/	该目录与 CPU_CDE 相同
-mycpu_axi_verify/	读者实现的类 SRAM 总线 CPU 的验证环境, 与 CPU_CDE 大部分类似
--rtl/	SoC_lite 设计代码目录
--soc_axi_lite_top.v	SoC_lite 的顶层文件
--myCPU/*	自己实现的 CPU 的 RTL 代码
--CONFREG/	confreg 模块, 用于访问 CPU 与开发板上数码管、拨码开关等外设

--ram_wrap/	以支持随机延迟访问封装的 AXI RAM 模块
--axi_wrap/	AXI 的 1x1 转接口, 连接 CPU 和 Crossbar, 用于抹平仿真和上板的差异
--xilinx_ip/	定制的 Xilinx IP, 包含 clk_pll、axi_ram 和 axi_crossbar_1x2
--testbench/	功能仿真验证平台
--mycpu_tb.v	功能仿真顶层, 该模块会抓取 debug 信息与 golden_trace.txt 进行比较
--run_vivado/	Vivado 工程的运行目录
--soc_lite.xdc	Vivado 工程设计的约束文件
--mycpu_prj1/	创建的第一个 Vivado 工程, 名字为 mycpu_prj1
--*.xpr	Vivado 创建的工程文件, 可直接打开
--*	创建的其他 Vivado 工程

请参考下列步骤完成本实践任务:

- 1) 请先完成本章的实践任务一。
- 2) 学习 8.3 节、8.4 节内容, 重点理解 AXI 总线协议的内容。
- 3) 将 lab11.zip 解压到路径上无中文字符的目录里, 实验环境为 CPU_CDE_AXI。
- 4) 解压 lab9 提供的 lab9.zip, 将 func_lab9/ 拷贝到 CPU_CDE_AXI/soft/ 目录里。
- 5) 打开 cpu132_gettrace 工程 (CPU_CDE_AXI/cpu132_gettrace/run_vivado/cpu132_gettrace/cpu132_gettrace.xpr)。
- 6) 对 cpu132_gettrace 工程中的 inst_ram 重新定制, 此时选择加载 func_lab9 的 coe (CPU_CDE_AXI/soft/func_lab9/obj/inst_ram.coe)。
- 7) 运行 cpu132_gettrace 工程的仿真 (进入仿真界面后, 直接点击 run all 等待仿真运行完成), 生成新的参考 Trace 文件 golden_trace.txt (CPU_CDE_SRAM/cpu132_gettrace/golden_trace.txt)。注意, 要等仿真运行完成后, golden_trace.txt 才有完整的内容。
- 8) 编写 RTL 代码, 将 lab10 完成的 myCPU 顶层接口包装成 AXI 接口。(推荐本任务中实现一个“类 SRAM-AXI”的 2x1 的转接桥, 然后拼接上实践任务一 (也就是 lab11) 完成的类 SRAM 接口的 CPU, 将 myCPU 封装为 AXI 接口。)
- 9) 将新完成的 myCPU 代码放在 CPU_CDE_AXI/mycpu_sram_verify/rtl/myCPU/ 目录里。
- 10) 打开 myCPU 工程 (CPU_CDE_AXI/mycpu_sram_verify/run_vivado/mycpu_prj1/mycpu_prj1.xpr)。
- 11) 通过“Add Sources”将新的 myCPU 源码添加到工程中。
- 12) 对 myCPU 工程中的 axi_ram 进行重新定制, 此时选择加载 func_lab9 的 coe (CPU_CDE_AXI/soft/func_lab9/obj/inst_ram.coe)。
- 13) 运行 myCPU 工程的仿真 (进入仿真界面后, 直接点击 run all), 开始调试。
- 14) 仿真通过后, 综合实现后生成比特流文件, 进行上板验证 (如果没有实验箱, 请跳过这一步)。在上板验证时, 要求 8 个拨码开关处于“高 4 个拨下, 低 4 个拨上”的状态 (对应访存随机延迟类型为无延迟), 能正确运行 func_lab9。不要求“随意切换拨码开关后按复位键”, 能正确运行 func_lab9。

8.5.3 实践任务三：完成 AXI 随机延迟验证

本实践任务的要求如下：使实践任务二中完成的添加 AXI 总线接口的 CPU 能通过随机延迟验证。

本实践任务的环境与实践任务二（也就是 lab11）相同。

请参考下列步骤完成本实践任务：

1) 请先完成本章的实践任务二。

2) 准备好 lab11 的实验环境 CPU_CDE_AXI。

3) 修改 CPU_CDE_SRAM/rtl/CONFREG/confreg.v 里的宏 RANDOM_SEED 的定义值，改为其他初始值，随后进行仿真和调试。

4) 重复第 3 步多次，要求宏 RANDOM_SEED 的修改值能覆盖本章前面介绍的三种随机延迟类型。

5) 仿真通过后，综合实现后生成比特流文件，进行上板验证（如果没有实验箱，请跳过这一步）。上板验证时，要求“随意切换拨码开关后按复位键”，能正确运行 func_lab9。如果出现“仿真通过，上板不过”的现象，请按照本章实践任务一给出的方法进行调试。